React Router

💧 1️⃣ What is React Router?

👉 React Router = **Page navigation system in React**

Without it:

- React has only **one page**

With it:

- You can create multiple pages:
  - `/` → Home
  - `/about` → About
  - `/contact` → Contact

💡 Works without page reload (SPA)

⚙️ 2️⃣ Installation

Open terminal inside your React project:

```
npm install react-router-dom
```

✅ That's it — package installed

3 Simple Project Structure

```
src/
 ├─ App.js
 ├─ Home.js
 ├─ About.js
 └─ Contact.js
```

4 Create Pages

Home.js

```
function Home() {
  return <h1>Home Page</h1>;
}
export default Home;
```

About.js

```
function About() {
  return <h1>About Page</h1>;
}
export default About;
```

Contact.js

```
function Contact() {
  return <h1>Contact Page</h1>;
}
export default Contact;
```

🚀 5️⃣ Main Routing Code (App.js)

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";
import Home from "./Home";
import About from "./About";
import Contact from "./Contact";

function App() {
  return (
    <BrowserRouter>

      {/* Navigation */}
      <nav style={{ display: "flex", gap: "20px" }}>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
        <Link to="/contact">Contact</Link>
      </nav>

      {/* Routes */}
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>

    </BrowserRouter>
  );
}

export default App;
```

🎯 6️⃣ How it Works (Simple)

👉 Step-by-step:

1. Click **Home / About / Contact**

2. URL changes (`/about`)
3. React finds matching `<Route>`
4. That component loads
5. Page updates **without refresh**

🧠 7️⃣ Important Concepts

Concept	Meaning
BrowserRouter	Enables routing
Routes	Group of routes
Route	One path → one component
Link	Navigation

⚡ One Line Summary

👉 **React Router = URL → Component mapping system**

🔥 What is Nested Routing?

Nested routing means:

👉 A **parent route (Dashboard)** contains **child routes (Profile, Settings)**

👉 UI is shared (Dashboard layout stays, content changes inside)

📁 Project Structure

```
src/
 ├── App.js
 ├── Dashboard.js
 ├── Profile.js
 ├── Settings.js
 ├── Home.js
 ├── index.js
```

🔵 1. index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

🔵 2. App.js

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Dashboard from "./Dashboard";
import Profile from "./Profile";
import Settings from "./Settings";
import Home from "./Home";
```

```
function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Home page */}
        <Route path="/" element={<Home />} />

        {/* Dashboard with nested routes */}
        <Route path="/dashboard" element={<Dashboard />}>
          <Route index element={<h2>Welcome to Dashboard</h2>} />
          <Route path="profile" element={<Profile />} />
          <Route path="settings" element={<Settings />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

3. Home.js

```
import { Link } from "react-router-dom";

function Home() {
  return (
    <div>
      <h1>Home Page</h1>

      <Link to="/dashboard">Go to Dashboard</Link>
    </div>
  );
}

export default Home;
```

4. Dashboard.js

```
import { Outlet, Link } from "react-router-dom";

function Dashboard() {
  return (
    <div>
      <h1>Dashboard</h1>

      {/* NAVBAR */}
      <nav style={{ display: "flex", gap: "20px" }}>
        <Link to="profile">Profile</Link>
        <Link to="settings">Settings</Link>
      </nav>

      <hr />

      {/* CHILD ROUTES RENDER HERE */}
      <Outlet />
    </div>
  );
}

export default Dashboard;
```

5. Profile.js

```
function Profile() {
  return (
    <div>
      <h2>Profile Page</h2>
    </div>
  );
}
```

```
export default Profile;
```

🟢 6. Settings.js

```
function Settings() {
  return (
    <div>
      <h2>Settings Page</h2>
    </div>
  );
}

export default Settings;
```

Dynamic Routing (React Router v6 – Advanced) 🚀

👉 Example: /users/1, /users/2 (dynamic user profile pages)

🗂 Project Structure

```
src/
 ├── App.js
 ├── Home.js
```

```
├── Users.js
├── UserDetails.js
├── index.js
```

1. index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

2. App.js

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import Users from "./Users";
import UserDetails from "./UserDetails";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Home */}
        <Route path="/" element={<Home />} />

        {/* Users list */}
        <Route path="/users" element={<Users />} />

        {/* Dynamic Route */}
        <Route path="/users/:id" element={<UserDetails />} />
      </Routes>
```

```
    </BrowserRouter>
  );
}

export default App;
```

3. Home.js

```
import { Link } from "react-router-dom";

function Home() {
  return (
    <div>
      <h1>Home Page</h1>

      <Link to="/users">Go to Users</Link>
    </div>
  );
}

export default Home;
```

4. Users.js (List Page)

```
import { Link } from "react-router-dom";

function Users() {
  const users = [
    { id: 1, name: "Anbu" },
    { id: 2, name: "Kumar" },
    { id: 3, name: "Ravi" }
  ];
```

```
  return (
    <div>
      <h1>Users List</h1>

      {users.map((user) => (
        <div key={user.id}>
          <Link to={`/users/${user.id}`}>
            {user.name}
          </Link>
        </div>
      ))}
    </div>
  );
}

export default Users;
```

🟢 5. UserDetails.js (Dynamic Page)

```
import { useParams, Link } from "react-router-dom";

function UserDetails() {
  const { id } = useParams(); // get dynamic ID

  return (
    <div>
      <h1>User Details</h1>

      <h2>User ID: {id}</h2>

      <Link to="/users">⬅ Back to Users</Link>
    </div>
  );
}
```

```
export default UserDetails;
```

🔥 What is Programmatic Navigation?

👉 Navigation using **JavaScript (not Link)**
👉 Used when:

- After login → go to dashboard
- After submit → redirect
- Button click → navigate

🗂 File Structure

```
src/
 ├── App.jsx
 ├── Home.jsx
 ├── Login.jsx
 ├── Dashboard.jsx
 ├── main.jsx
```

🟣 1. App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Home from "./Home";
import Login from "./Login";
import Dashboard from "./Dashboard";

function App() {
  return (
    <BrowserRouter>
      <Routes>
```

```
        <Route path="/" element={<Home />} />
        <Route path="/login" element={<Login />} />
        <Route path="/dashboard" element={<Dashboard />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

2. Home.jsx

```
import { useNavigate } from "react-router-dom";

function Home() {
  const navigate = useNavigate();

  return (
    <div>
      <h1>Home Page</h1>

      <button onClick={() => navigate("/login")}>
        Go to Login
      </button>
    </div>
  );
}

export default Home;
```

3. Login.jsx

```
import { useNavigate } from "react-router-dom";

function Login() {
  const navigate = useNavigate();
```

```
  const handleLogin = () => {
    // fake login logic
    alert("Login Successful");

    // navigate to dashboard
    navigate("/dashboard");
  };

  return (
    <div>
      <h1>Login Page</h1>

      <button onClick={handleLogin}>
        Login
      </button>
    </div>
  );
}

export default Login;
```

4. Dashboard.jsx

```
import { useNavigate } from "react-router-dom";

function Dashboard() {
  const navigate = useNavigate();

  return (
    <div>
      <h1>Dashboard</h1>

      <button onClick={() => navigate("/")}>
        Go Home
      </button>

      <button onClick={() => navigate(-1)}>
```

```
        Go Back
      </button>
    </div>
  );
}

export default Dashboard;
```

🟣 5. main.jsx (Vite)

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <App />
);
```

---

404 Page (Not Found) setup in React Router v6 🚀

🗂 File Structure

```
src/
 ├── App.jsx
 ├── Home.jsx
 ├── NotFound.jsx
 ├── main.jsx
```

1. App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-
dom";
import Home from "./Home";
import NotFound from "./NotFound";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Valid Route */}
        <Route path="/" element={<Home />} />

        {/* 404 Route (MUST BE LAST) */}
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

2. Home.jsx

```
import { Link } from "react-router-dom";

function Home() {
  return (
    <div>
      <h1>Home Page</h1>

      <Link to="/abc">Go to Wrong Page</Link>
    </div>
```

```
  );
}

export default Home;
```

3. NotFound.jsx

```
import { Link } from "react-router-dom";

function NotFound() {
  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h1>404</h1>
      <h2>Page Not Found</h2>

      <Link to="/">Go Home</Link>
    </div>
  );
}

export default NotFound;
```

4. main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <App />
);
```

🔥 How It Works

```
<Route path="*" element={<NotFound />} />
```

👉 * = catches **all invalid URLs**

👉 Example:

- `/abc`
- `/xyz`
- `/wrong/page`

➡️ All go to **404 Page**

Protected Routes (Advanced – React Router v6) 🔐

👉 Includes: login check, redirect, nested protection, role-based access

🗂 File Structure

```
src/
 ├── App.jsx
 ├── Login.jsx
 ├── Dashboard.jsx
 ├── Admin.jsx
 ├── ProtectedRoute.jsx
 ├── main.jsx
```

1. App.jsx

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Login from "./Login";
import Dashboard from "./Dashboard";
import Admin from "./Admin";
import ProtectedRoute from "./ProtectedRoute";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Public */}
        <Route path="/" element={<Login />} />

        {/* Protected Routes */}
        <Route element={<ProtectedRoute />}>
          <Route path="/dashboard" element={<Dashboard />} />
        </Route>

        {/* Role-based Protected */}
        <Route element={<ProtectedRoute role="admin" />}>
          <Route path="/admin" element={<Admin />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

2. ProtectedRoute.jsx

```
import { Navigate, Outlet } from "react-router-dom";

function ProtectedRoute({ role }) {
  const isAuth = localStorage.getItem("isAuth"); // true/false
  const userRole = localStorage.getItem("role"); // "admin" or "user"

  // Not logged in
```

```
  if (!isAuth) {
    return <Navigate to="/" replace />;
  }

  // Role check (if required)
  if (role && userRole !== role) {
    return <Navigate to="/dashboard" replace />;
  }

  return <Outlet />;
}

export default ProtectedRoute;
```

3. Login.jsx

```
import { useNavigate } from "react-router-dom";

function Login() {
  const navigate = useNavigate();

  const handleLogin = () => {
    // fake login
    localStorage.setItem("isAuth", "true");
    localStorage.setItem("role", "admin"); // change to "user" to test

    navigate("/dashboard", { replace: true });
  };

  return (
    <div>
      <h1>Login Page</h1>

      <button onClick={handleLogin}>
        Login as Admin
      </button>
    </div>
  );
}
```

```
export default Login;
```

4. Dashboard.jsx

```
import { useNavigate } from "react-router-dom";

function Dashboard() {
  const navigate = useNavigate();

  const logout = () => {
    localStorage.removeItem("isAuth");
    localStorage.removeItem("role");
    navigate("/", { replace: true });
  };

  return (
    <div>
      <h1>Dashboard (Protected)</h1>

      <button onClick={() => navigate("/admin")}>
        Go to Admin
      </button>

      <button onClick={logout}>
        Logout
      </button>
    </div>
  );
}

export default Dashboard;
```

5. Admin.jsx

```
function Admin() {
  return (
    <div>
      <h1>Admin Page (Only Admin Allowed)</h1>
```

```
    </div>
  );
}

export default Admin;
```

6. main.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")).render(
  <App />
);
```

How It Works

Step 1: User logs in

```
localStorage.setItem("isAuth", "true");
localStorage.setItem("role", "admin");
```

Step 2: ProtectedRoute checks

```
if (!isAuth) → redirect to login
if role mismatch → redirect
```

Step 3: Access control

Route	Access
`/dashboard`	Logged-in users
`/admin`	Only admin

🧠 Key Concepts

🔑 `<Outlet />`

👉 Renders protected child routes

🔑 `<Navigate />`

👉 Redirect user

🔑 Role-based

```
<ProtectedRoute role="admin" />
```

🚀 Advanced Tips

👉 Use **JWT token instead of localStorage (real apps)**
👉 Store auth in **context (AuthContext)**
👉 Show loading spinner while checking auth

⚡ Flow

```
Login → set auth → access dashboard
Not logged → redirect login
Wrong role → redirect dashboard
```